实验 1：基于 Paillier 算法实现隐私信息获取
数据安全第一次实验报告

姓名：郭佳成　学号：2311990　专业：密码科学与技术

代码：https://github.com/Fighting05/DataSecurity

2026 年 3 月 24 日

1 实验要求

1. 基于 Paillier 算法实现隐私信息获取：从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获取了哪一个消息，同时客户端能完成获取消息的解密。
2. 实现 Paillier 密码算法，包括密钥生成、加密和解密函数。
3. 利用 Paillier 的加法同态和数乘同态性质实现 1-out-of-N 不经意传输。
4. 程序结构清晰，具有较好的可读性。
5. 提交程序源码和实验报告。
6. **扩展实验**：有能力的同学可以在客户端保存对称密钥 k，在服务器端存储 m 个用对称密钥 k 加密的密文，通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

2 实验环境

- **操作系统**：Windows 11
- **开发语言**：Python 3.9.12
- **依赖库**：phe（Python Paillier 加密库）、cryptography（AES 对称加密库）

3 实验原理

3.1 Paillier 密码算法概述

Paillier 密码系统是由 Pascal Paillier 在 1999 年提出的一种概率公钥加密算法，具有加法同态性质，广泛应用于安全多方计算、电子投票、隐私信息获取等场景。

3.2 Paillier 算法详细步骤

3.2.1 密钥生成

1. 选择两个大素数 p 和 q，满足 $\gcd(pq, (p-1)(q-1)) = 1$
2. 计算 $n = p \times q$
3. 计算 $\lambda = \mathrm{lcm}(p-1, q-1)$（最小公倍数）
4. 选择 $g = n + 1$（简化选择，自动满足条件）
5. 计算 $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$，其中 $L(x) = \frac{x-1}{n}$

公钥：(n, g)
私钥：(λ, μ)

3.2.2 加密算法

给定明文 $m \in \mathbb{Z}_n$：

1. 选择随机数 $r \in \mathbb{Z}_n^*$（即 $0 < r < n$ 且 $\gcd(r, n) = 1$）
2. 计算密文：$c = g^m \cdot r^n \bmod n^2$

3.2.3 解密算法

给定密文 $c \in \mathbb{Z}_{n^2}^*$：

1. 计算：$m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n$

3.3 Paillier 的同态性质

Paillier 密码系统具有**加法同态**特性，这是实现隐私信息获取的关键。

3.3.1 加法同态

给定两个密文 $c_1 = \text{Enc}(m_1)$，$c_2 = \text{Enc}(m_2)$，则：

$$c_1 \cdot c_2 \bmod n^2 = \text{Enc}(m_1 + m_2 \bmod n)$$

即：**密文相乘 = 明文相加**

3.3.2 数乘同态（标量乘法）

给定密文 $c = \text{Enc}(m)$ 和常数 k，则：

$$c^k \bmod n^2 = \text{Enc}(k \cdot m \bmod n)$$

即：**密文求 k 次方 = 明文乘以 k**

关键性质：

- $\text{Enc}(0) \times k = \text{Enc}(0)$（0 乘以任何数都是 0）
- $\text{Enc}(1) \times k = \text{Enc}(k)$（1 乘以 k 就是 k）
- $\text{Enc}(a) + \text{Enc}(b) = \text{Enc}(a + b)$（密文相加即明文相加）

这些性质是实现隐私信息获取的核心！

3.4 基于 Paillier 的隐私信息获取协议

3.4.1 场景描述

- 服务器持有 m 个消息 $M_1, M_2, \ldots, M_m$
- 客户端想获取第 i 个消息 M_i
- 要求：服务器不知道客户端选择了哪个 i；客户端只能得到 M_i

3.4.2 协议流程

准备阶段：

1. 客户端运行 Paillier 密钥生成，得到公钥和私钥
2. 客户端将公钥发送给服务器

查询阶段：

1. 客户端选择要获取的消息索引 i

2. 客户端生成向量 $\{v_1, v_2, \ldots, v_m\}$，其中：
   - $v_i = \text{Enc}(1)$（加密 1）
   - $v_j = \text{Enc}(0)$，当 $j \neq i$（加密 0）
3. 客户端将向量发送给服务器

响应阶段：

1. 服务器接收到 m 个密文 $v_1, v_2, \ldots, v_m$，对于服务器的 m 个明文 $p_1, p_2, \ldots, p_m$，计算：
$$C = v_1 \times p_1 + v_2 \times p_2 + \cdots + v_m \times p_m$$
2. 假设客户端要获取的明文为 p_k，上述式子利用加法同态和标量乘法化简为：
$$\text{Enc}(0) \times p_1 + \cdots + \text{Enc}(1) \times p_k + \cdots + \text{Enc}(0) \times p_m = \text{Enc}(0) + \text{Enc}(p_k) = \text{Enc}(p_k)$$
3. 服务器将 $C = \text{Enc}(p_k)$ 发送给客户端

解密阶段：

1. 客户端使用私钥解密 C，得到 M_i

安全性保证：

- 服务器视角：看到的全是密文，无法区分 Enc(0) 和 Enc(1)，因此不知道客户端选了哪个
- 客户端视角：只能解密出 $\text{Enc}(p_k)$，无法得到其他消息

4 准备阶段

4.1 环境准备阶段

为了实现此次实验我们选用 **Python** 环境，因此需要安装 phe 库来支持 Paillier。

可以看到下图展示了我们的环境准备过程，安装 phe 库后导入成功，环境准备好了，接下来我们来尝试一下使用 Paillier。

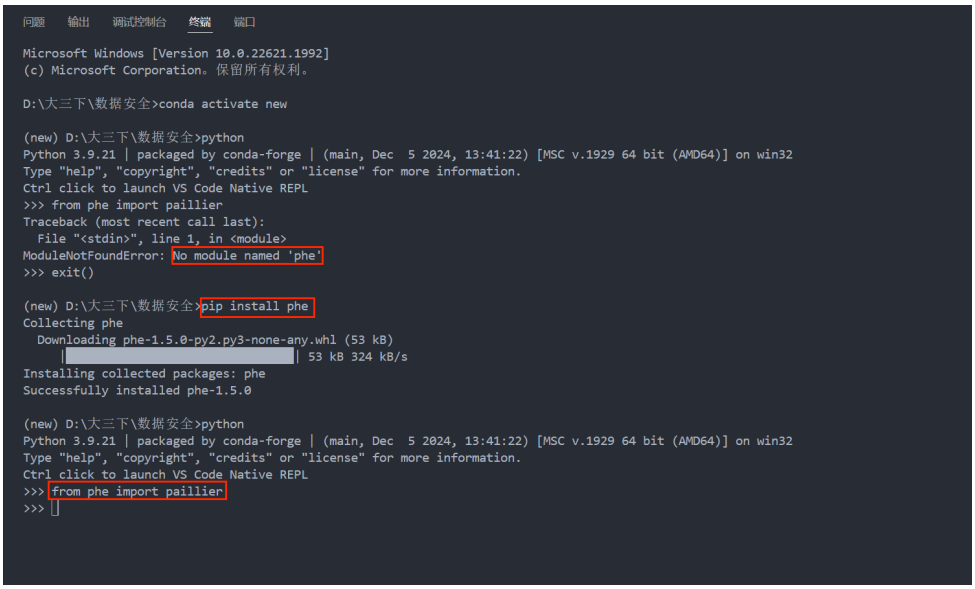

图 1: 环境准备

4.2 测试代码-基于 Python 的 phe 库完成加法和标量乘法的验证

4.2.1 简单演示代码

借用了书上面的示例代码。主要完成以下的一些验证工作，作为前期准备。

1. 成功验证并使用了他的加解密
2. 成功使用了被重载的计算符号比如“+”，“-”，“/”都是经过了 Paillier 库重载的。
3. 最重要的验证了其所具有的加法同态，以及确实不支持乘法同态（可以看一下代码运行最后的报错）

```python
from phe import paillier  # 开源库
import time  # 做性能测试

#################### 设置参数
print("默认私钥大小: ", paillier.DEFAULT_KEYSIZE)
# 生成公私钥
public_key, private_key = paillier.generate_paillier_keypair()
# 测试需要加密的数据
message_list = [3.1415926, 100, -4.6e-12]

#################### 加密操作
time_start_enc = time.time()
encrypted_message_list = [public_key.encrypt(m) for m in message_list
    ]
```

```python
time_end_enc = time.time()
print("加密耗时s: ", time_end_enc - time_start_enc)
print("加密数据（3.1415926）:", encrypted_message_list[0].ciphertext
    ())

#################### 解密操作
time_start_dec = time.time()
decrypted_message_list = [private_key.decrypt(c) for c in
    encrypted_message_list]
time_end_dec = time.time()
print("解密耗时s: ", time_end_dec - time_start_dec)
print("原始数据（3.1415926）:", decrypted_message_list[0])

#################### 测试加法和乘法同态
a, b, c = encrypted_message_list  # a,b,c分别为对应密文
a_sum = a + 5  # 密文加明文，已经重载了+运算符
a_sub = a - 3  # 密文加明文的相反数，已经重载了-运算符
b_mul = b * 6  # 密文乘明文,数乘
c_div = c / -10.0  # 密文乘明文的倒数

print("a+5=", private_key.decrypt(a_sum))
print("a-3", private_key.decrypt(a_sub))
print("b*6=", private_key.decrypt(b_mul))
print("c/-10.0=", private_key.decrypt(c_div))

## 密文加密文
print((private_key.decrypt(a) + private_key.decrypt(b)) ==
    private_key.decrypt(a + b))
# 报错，不支持a*b，即两个密文直接相乘
try:
    print((private_key.decrypt(a) + private_key.decrypt(b)) ==
        private_key.decrypt(a * b))
except Exception as e:
    print("a*b␣报错，不支持两个密文直接相乘:", e)
```

Listing 1: Paillier 加解密和同态性质测试代码

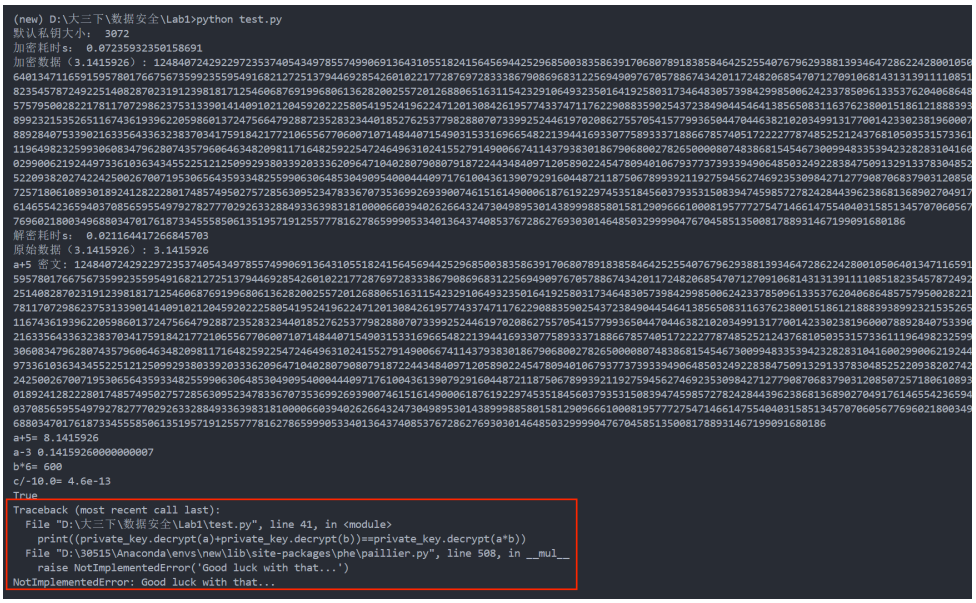

图 2: 验证算法

5　程序设计与实现

5.1　代码实现

由于实验比较简单，如何设计这样的隐私获取在 3.4 节我们已经给出了，这是最重要的部分。因此代码架构就不复杂化，我们将客户端以及服务器都在一个代码中模拟给出。

5.2　主实验代码

基于 phe 库，我们实现完整的隐私信息获取流程：

```python
from phe import paillier

def main():
    print("=" * 60)
    print("    基于Paillier算法的隐私信息获取实验")
    print("=" * 60)

    # 服务器持有m个消息
    server_messages = [
        "Hello, World!",
        "数据安全实验",
        "Paillier同态加密",
        "隐私信息获取",
```

```
        "1-out-of-N不经意传输"
    ]
    m = len(server_messages)
    print(f"\n服务器持有 {m} 条消息")

    # 客户端：密钥生成
    public_key, private_key = paillier.generate_paillier_keypair()
    print("\n客户端：密钥生成完成")

    # 客户端选择要获取的消息
    target_index = 2
    print(f"\n客户端选择获取第 {target_index+1} 条消息")

    # 生成密文向量：enc(0) 和 enc(1)
    encrypted_vector = []
    for i in range(m):
        if i == target_index:
            encrypted_vector.append(public_key.encrypt(1))
        else:
            encrypted_vector.append(public_key.encrypt(0))
    print("\n客户端：密文向量生成完成")

    # 消息转整数的函数
    def message_to_int(msg):
        return sum(ord(c) for c in msg)

    def int_to_message(value, original_messages):
        for msg in original_messages:
            if message_to_int(msg) == value:
                return msg
        return None

    # 服务器端同态计算
    result = None
    for i in range(m):
        msg_int = message_to_int(server_messages[i])
        term = encrypted_vector[i] * msg_int
        result = term if result is None else result + term
    print("\n服务器：同态计算完成")

```

```
    # 客户端解密
    decrypted_int = private_key.decrypt(result)
    decrypted_message = int_to_message(decrypted_int, server_messages
        )

    # 验证
    print("\n" + "=" * 60)
    expected_message = server_messages[target_index]
    if decrypted_message == expected_message:
        print(f"成功！获取到消息:␣{decrypted_message}")
    print("=" * 60)


if __name__ == "__main__":
    main()
```

Listing 2: 基于 Paillier 的隐私信息获取主代码（精简版）

5.3 扩展实验：AES 对称加密 + Paillier

扩展实验的设计思路：客户端保存 AES 对称密钥，服务器存储用 AES 加密后的消息。通过 Paillier 隐私获取方法得到指定密文后，客户端用 AES 密钥解密得到明文。

5.3.1 扩展实验代码

```
from phe import paillier
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
    , modes
from cryptography.hazmat.primitives import padding
import os
import base64


class AESCipher:
    def __init__(self, key=None):
        self.key = key if key else os.urandom(32)

    def encrypt(self, plaintext):
        data = plaintext.encode('utf-8')
        padder = padding.PKCS7(128).padder()
```

```python
        padded = padder.update(data) + padder.finalize()
        iv = os.urandom(16)
        cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
        enc = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
        return base64.b64encode(iv + enc).decode('utf-8')

    def decrypt(self, enc_str):
        data = base64.b64decode(enc_str.encode('utf-8'))
        iv, enc = data[:16], data[16:]
        cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
        padded = cipher.decryptor().update(enc) + cipher.decryptor().finalize()
        unpadder = padding.PKCS7(128).unpadder()
        return (unpadder.update(padded) + unpadder.finalize()).decode('utf-8')


def main():
    print("=" * 60)
    print("    扩展实验: AES + Paillier")
    print("=" * 60)

    # 客户端: 生成AES密钥
    aes = AESCipher()
    print("\n[客户端] 生成AES密钥")

    # 原始消息
    messages = ["消息1", "机密数据", "隐私信息", "测试内容"]
    print(f"\n原始消息: {messages}")

    # 客户端: AES加密所有消息
    enc_msgs = [aes.encrypt(m) for m in messages]
    print("\n[客户端] 用AES加密消息完成")

    # 服务器: 存储加密消息
    server_data = enc_msgs.copy()
    print("\n[服务器] 存储加密消息")

    # Paillier密钥生成
```

```python
    pubkey, privkey = paillier.generate_paillier_keypair()

    # 客户端选择消息索引
    target_idx = 1
    print(f"\n[客户端] 选择获取第 {target_idx+1} 条消息")

    # 生成密文向量
    vec = []
    for i in range(len(messages)):
        vec.append(pubkey.encrypt(1 if i == target_idx else 0))

    # 服务器：同态计算得到索引
    result = None
    for i in range(len(messages)):
        term = vec[i] * (i + 1)
        result = term if result is None else result + term
    print("\n[服务器] 同态计算完成")

    # 客户端解密得到索引
    idx = privkey.decrypt(result) - 1
    print(f"\n[客户端] 解密得到索引: {idx}")

    # 获取并解密消息
    final = aes.decrypt(server_data[idx])
    print(f"\n[客户端] AES解密得到: {final}")

    print("\n" + "=" * 60)
    print("    扩展实验完成！")
    print("=" * 60)


if __name__ == "__main__":
    main()
```

Listing 3: 扩展实验：AES + Paillier

5.4 字符串消息处理说明

由于 Paillier 加密只能处理整数，对于字符串消息，我们需要进行编码转换。上述代码使用了简单的字符求和作为演示，实际应用中可以采用以下方式：

1. 将消息分成固定大小的块，每块编码为一个整数
2. 使用对称密钥加密消息，然后用 Paillier 加密对称密钥
3. 采用混合加密方案

6 运行与测试

6.1 功能测试

6.1.1 隐私信息获取测试

测试目的：验证客户端能正确获取指定消息，且服务器无法知道客户端的选择。

- **测试方法：**
  1. 服务器准备 5 条测试消息
  2. 客户端依次选择获取第 1、3、5 条消息
  3. 验证每次都能正确获取到对应的消息
- **预期结果：**客户端每次都能正确获取到指定索引的消息
- **测试结果：**

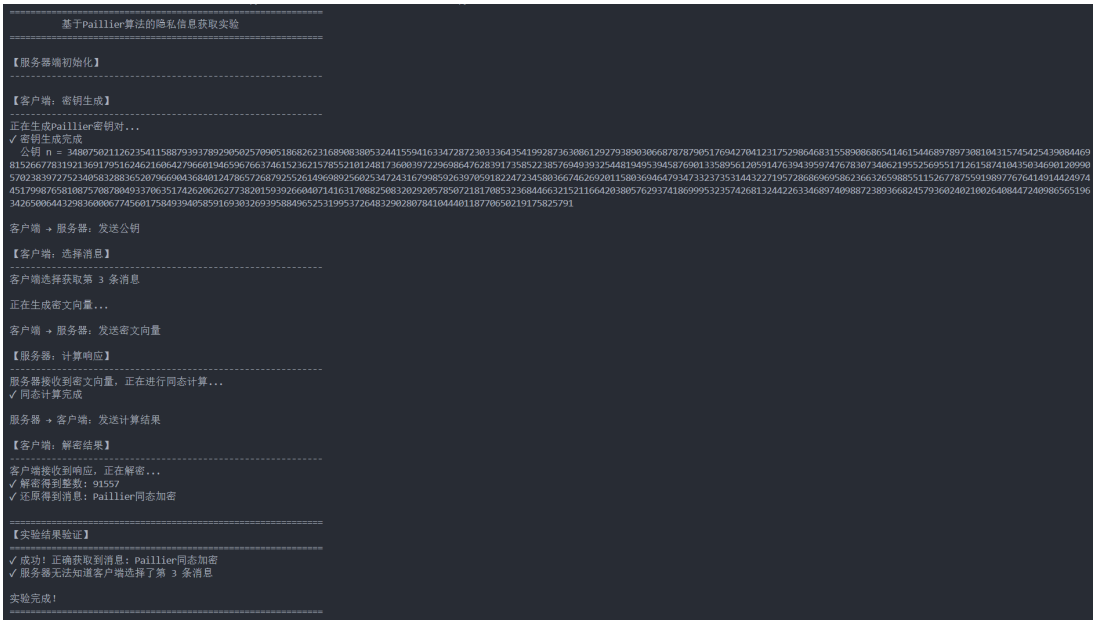

图 3: 获取第 3 条消息测试, 其余同理

6.1.2 扩展实验设计与实现

扩展实验结合对称加密和同态加密的优势，采用混合加密方案：

1. 客户端生成 AES 对称密钥 k 并保存
2. 用 k 加密所有消息，上传加密后的消息给服务器
3. 使用 Paillier 隐私获取方法，从服务器获取指定的加密消息
4. 客户端用 k 解密得到原始消息

方案优势：AES 效率高，Paillier 保证选择隐私，服务器无法获知原始内容。

实现要点：使用 AES-256-CBC 模式，随机 IV 保证语义安全，通过 Paillier 获取索引后请求对应密文。

```
============================================================

D:\大三下\数据安全>D:/30515/Anaconda/envs/new/python.exe d:/大三下/数据安全/Lab1/extended.py
============================================================
  扩展实验: AES + Paillier
============================================================

[客户端] 生成AES密钥

原始消息: ['消息1', '机密数据', '隐私信息', '测试内容']

[客户端] 用AES加密消息

[服务器] 存储加密消息

[客户端] 生成Paillier密钥

[客户端] 选择获取第 2 条消息

[服务器] 同态计算

[客户端] 解密得到索引: 1

[客户端] AES解密得到: 机密数据

============================================================
  扩展实验完成!
============================================================
```

图 4: 扩展实验运行结果：AES + Paillier

6.2 安全性分析

6.2.1 对服务器的隐私性

- 服务器看到的是加密的向量 $\{v_j\}$
- 由于 Paillier 是语义安全的，服务器无法区分 Enc(0) 和 Enc(1)
- 因此服务器无法知道客户端选择的索引 i

6.2.2 对客户端的安全性

- 客户端只能得到 M_i，无法获取其他消息 $M_j (j \neq i)$
- 所有消息都经过同态运算混合，客户端只能解密出自己选择的那条

正确性总结：

- 服务器无法区分客户端发来的密文是 Enc(0) 还是 Enc(1)
- 因此服务器无法得知客户端想要获取哪一个信息
- 同时客户端最后只会得到他要的那个信息，无法得到其他多余的信息

7 总结

本次实验成功实现了基于 Paillier 同态加密的 1-out-of-N 隐私信息获取系统，并完成了扩展实验。

主实验：利用 phe 库实现 Paillier 密钥生成、加密和解密，通过加法同态和数乘同态完成隐私信息获取，验证了服务器无法区分 Enc(0) 和 Enc(1)，客户端能正确解密得到目标消息。

扩展实验：实现 AES-256-CBC 加密，客户端保存密钥，服务器存储加密消息，通过 Paillier 获取指定密文后用 AES 解密，验证了混合加密方案的安全性。

通过实验，深入理解了 Paillier 同态加密原理，掌握了混合加密方案的设计思想，体会到同态加密在隐私保护领域的应用价值。

附录：运行说明

环境要求

- Windows 11 操作系统
- Python 3.9 或更高版本
- phe 库（Python Paillier 加密库）

安装依赖

```
pip install phe cryptography
```

运行步骤

1. 将实验代码保存为 `main.py`
2. 在命令行中运行 `python main.py`
3. 观察程序输出，验证隐私信息获取是否成功
4. 可修改代码中的 `target_index` 变量来测试获取不同消息

文件清单

- `main.py` - 主实验代码（包含客户端和服务器模拟）
- `extended.py` - 扩展实验代码（AES + Paillier）
- `test.py` - Paillier 加解密和同态性质测试代码